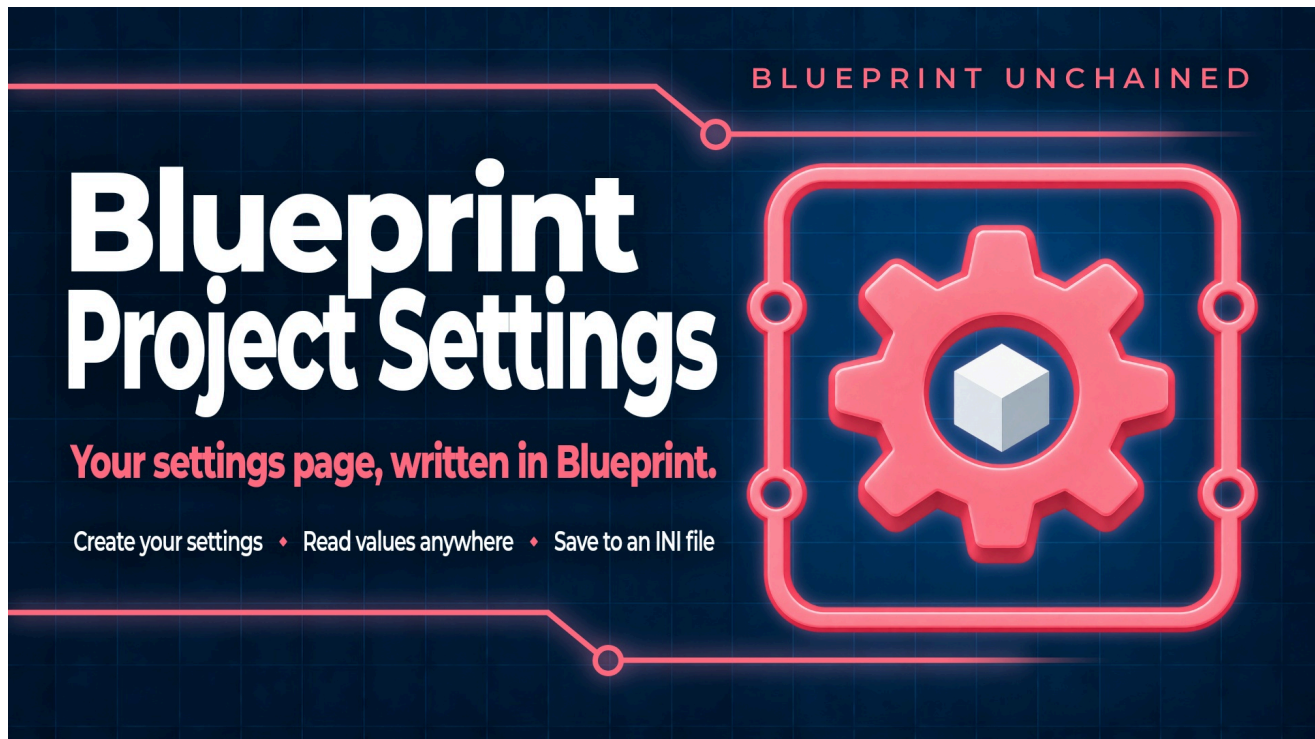


Blueprint Project Settings



Author: **Tomasz Klin**

Version: **1.0.2**

Contents

Introduction

Your first settings page

1. Create the settings Blueprint
2. Add the variables
3. Name the page
4. Compile and save
5. Read the values in any Blueprint

Beyond the basics

What the ini files give you
Where the values are stored
Pages for your editor tools
Saving from a Blueprint
The section name
The page

Settings that build on other settings

Reset to Defaults

Reacting to values

Config variables on any object

Packaging

Troubleshooting

The page is not in the settings panel

Get Settings has no variables to drag off

Save Settings returns false in the editor

My settings work in the editor and are missing from a build

A level opened by name is missing from a build

Saving does nothing in a packaged game

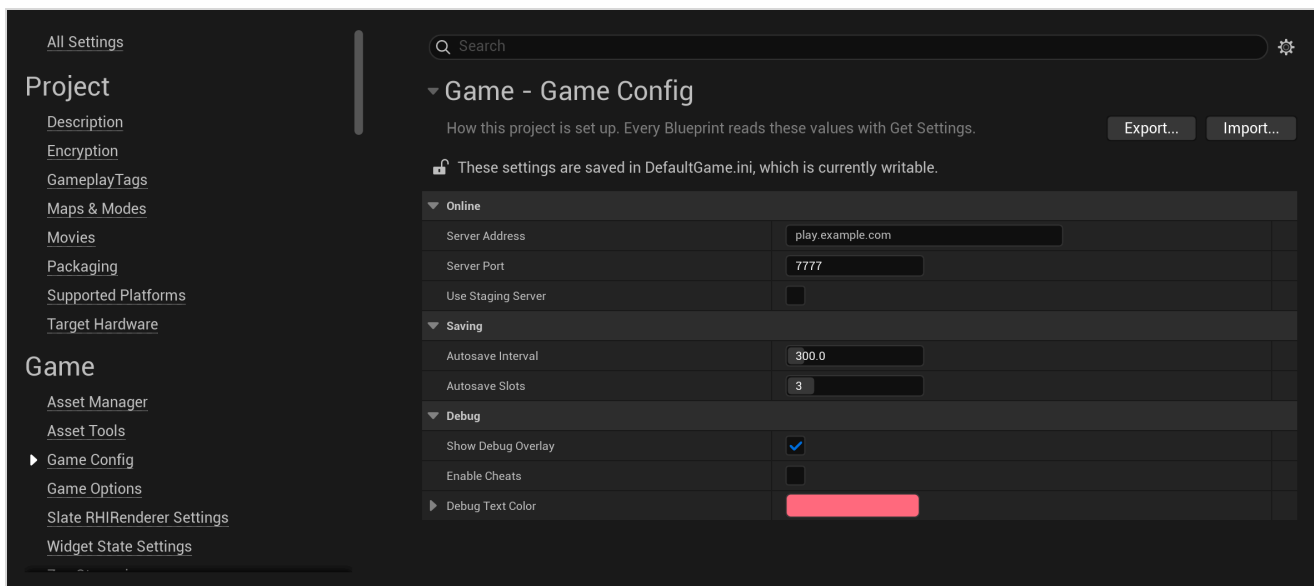
Values are back to the defaults after I moved the asset

Reset to Defaults does nothing
Support

Introduction

Blueprint Project Settings lets you create pages in **Project Settings** and **Editor Preferences** using Blueprint. Add variables to a settings Blueprint, compile it and read the values with **Get Settings**. No C++ is required.

Use a project settings page for values such as a server address, an autosave interval or debug options. Use Editor Preferences for your own tools, such as an export folder or a placement grid size. The values are stored in standard Unreal Engine ini files.



Start with [Your first settings page](#). [Beyond the basics](#) explains storage, editor tools, saving and packaging.

Your first settings page

This example creates a page called **Game Config**. It holds connection settings, save options and debug options for a small online game.

1. Create the settings Blueprint

Enable **Blueprint Project Settings** in **Edit > Plugins**. Restart the editor if prompted.

In the Content Browser, choose **Add > Blueprint > Blueprint Project Settings**. The asset is created at once, with its name ready to type: name it `BP_GameConfig`.

2. Add the variables

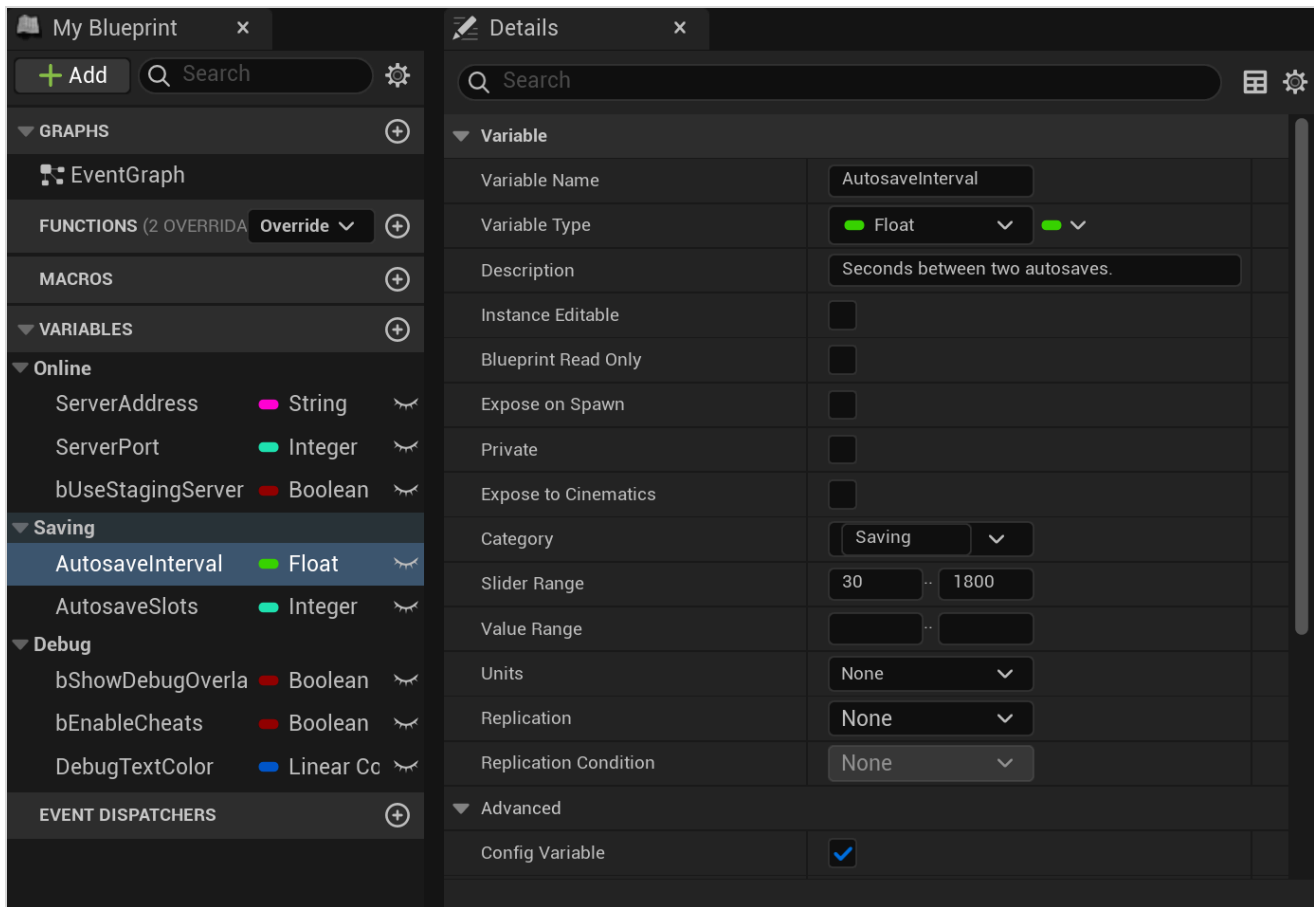
Open `BP_GameConfig` and add the variables below. In each variable's details, open **Advanced** and enable **Config Variable**. This marks the variable as a setting to store in an ini file.

Variable	Type	Category	Default value
ServerAddress	String	Online	play.example.com
ServerPort	Integer	Online	7777
bUseStagingServer	Boolean	Online	unticked
AutosaveInterval	Float	Saving	300
AutosaveSlots	Integer	Saving	3
bShowDebugOverlay	Boolean	Debug	ticked
bEnableCheats	Boolean	Debug	unticked
DebugTextColor	Linear Color	Debug	a red

The variable's details also control how it appears on the page:

- **Category** groups related settings.
- **Description** provides a tooltip when you hover over a setting.
- **Slider Range** sets the range for a number's slider.

Set each starting value in **Class Defaults** or in the variable's **Default Value** field.



3. Name the page

In **Class Defaults**, open **Settings Page**:

1. Set **Settings Display Name** to `Game Config`.
2. Enter a short **Settings Description**. It appears below the page title.
3. Choose a **Settings Category**. The default is `Game`; a new name creates a new group.

If the display name is empty, the page uses its settings section name. You can keep the default **Settings Storage** options for this example. The values will be stored in `Config/DefaultGame.ini`.

4. Compile and save

Compile and save `BP_GameConfig`. Open **Project Settings > Game > Game Config**. Your variables appear in their categories with the values you set.

Changes made on this page are saved to `Config/DefaultGame.ini`. For example, after changing the autosave interval to 600, the section contains:

```
ini [BP_GameConfig] ServerAddress=play.example.com ServerPort=7777
bUseStagingServer=False AutosaveInterval=600.000000 AutosaveSlots=3
```

```
bShowDebugOverlay=True bEnableCheats=False DebugTextColor=
(R=1.000000,G=0.147027,B=0.219526,A=1.000000) ``
```

Keep this file in source control to share the settings with your team. These project defaults are also included when you package the game.

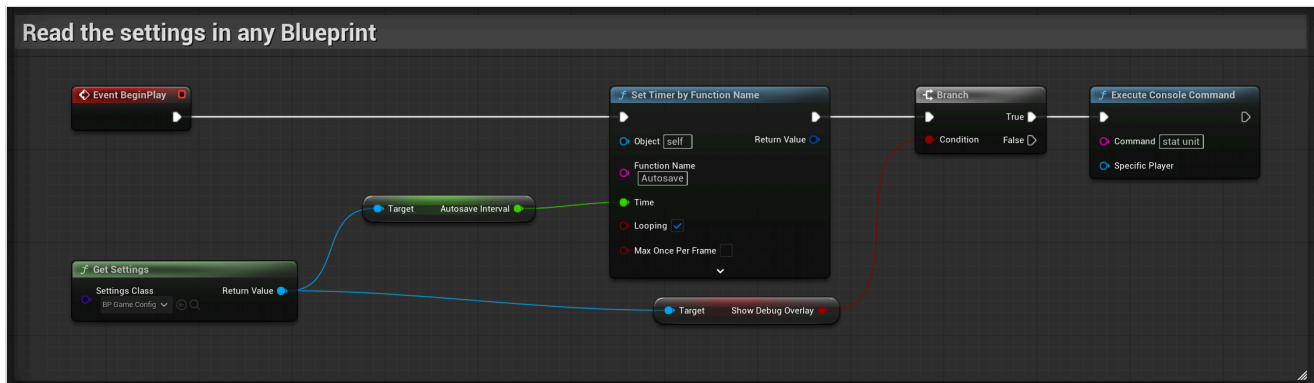
Changing a setting on the page does not change the Blueprint's original defaults. The page's **Reset to Defaults** button restores the values set in **Class Defaults**.

5. Read the values in any Blueprint

1. Right-click in a graph, type `Get Settings` and press **Enter**.
2. Select `BP_GameConfig` in **Settings Class**.
3. Drag from **Return Value** and choose the variable you want to read.

The node returns the selected Blueprint's type, so you do not need a cast.

The game mode below reads **AutosaveInterval** to start its autosave timer. It also reads **bShowDebugOverlay** to decide whether to show frame timings.



Each settings Blueprint has one shared settings object. Every Get Settings node for that class returns the same object. You do not need to spawn it or pass references between Blueprints.

Get Settings works in actors, widgets, game modes, Editor Utility Widgets and Animation Blueprints. It does not need a World or Game Instance. Stored values are loaded before Blueprint gameplay starts.

Beyond the basics

What the ini files give you

Ini files are plain text files used by Unreal Engine for configuration.

- **Edit outside the editor.** Open the file in a text editor and change a value. The project reads the updated values the next time it starts.
- **Review changes with your team.** Source control shows which values changed. Changes can be compared and merged as text.
- **Keep personal values.** A developer can override project values in `Config/UserGame.ini`. Keep that personal file out of source control.
- **Set platform-specific values.** For example, use `Config/Android/AndroidGame.ini` to override values for Android.

Where the values are stored

Two options in **Class Defaults > Settings Storage** control storage:

- **Config File** selects the file group. Use **Game** for most project settings. **Engine** and **Input** store settings in those engine config groups. The editor groups are described under [Pages for your editor tools](#). **Custom** lets you choose your own config file name.
- **Storage** selects which copy of that file receives saved changes.

The following paths are examples for **Config File = Game**:

Storage	File	Use
Project Default	<code>Config/DefaultGame.ini</code>	Shared project values. Keep in source control and include with the game.
Project User	<code>Config/UserGame.ini</code>	Personal values for one developer in this project. Keep out of source control.
Global User	The user settings directory	Personal values for one developer across projects on this machine.
Saved	<code>Saved/Config/<Platform>/Game.ini</code>	Values for this installation that a running game can write.

Use **Project Default** for shared project settings. A packaged game can read those defaults but cannot save changes back to the project's source file. In that case, **Save Settings** returns false and reports the reason in the log.

Use **Saved** for values that a packaged game needs to change. Editor config files are available only in the editor.

Stored In shows the exact path selected by Config File and Storage.

▼ Settings Page		
Settings Display Name	Game Config	↩
Settings Description	How this project is set up. Every Blueprint reads these values with Get Settings.	↩
Container	Project	
Settings Category	Game	
Show a page in the settings panel	☒	
▼ Settings Storage		
Config File	Game	
Storage	Project Default	
Section Id	GameConfig	↩
Stored In	./Config/DefaultGame.ini	

Pages for your editor tools

Set **Container** to **Editor Preferences** to create a page for your editor tools. Choose a category for the page, including a new category of your own.

For example, the page can hold an export folder, a placement grid size or a highlight colour.

[Blueprint Quaternion](#)
[Camera Calibration Editor](#)
[Dataflow Editor](#)
[Fracture Mode](#)
[Gameplay Cameras Editor](#)
[Light Mixer](#)
[Modeling Mode](#)
[Modeling Mode Tools](#)
[Object Mixer](#)
[Python](#)
[Rewind Debugger](#)
[StateTree Editor](#)
[Visual Studio](#)

Studio Tools

► Level Design

Q Search

▼ Studio Tools - Level Design

Your own preferences for the studio's level design tools. Kept out of the project.

Export... Import...

▼ Placement

Grid Snap Size

10

Align to Surface

☒

Random Yaw

☒

Scale Variation

0.15

▼ Viewport

Show Hints

☒

► Highlight Color

Label Size

14

Choose storage to match who should use the values:

- **Personal values for this project:** set Config File to **Editor Per Project User Settings** and Storage to **Saved**.
- **Personal values across projects:** use **Editor Per Project User Settings** with **Global User**.
- **Shared team values:** use **Editor** with **Project Default**. Store `Config/DefaultEditor.ini` in source control.

Keep personal settings files out of source control.

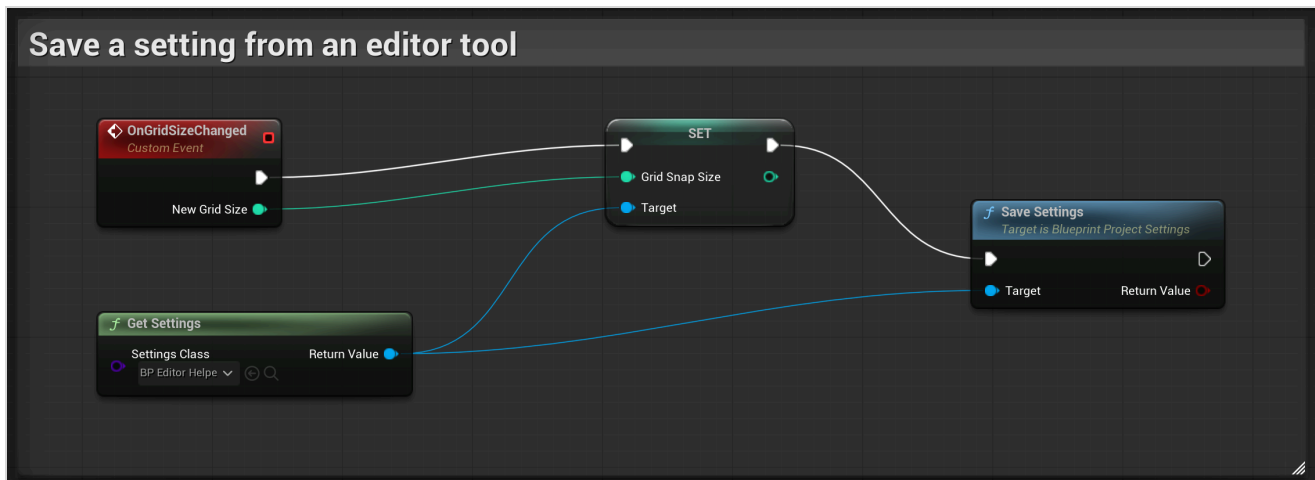
Editor Utility Widgets and editor scripts read these values with **Get Settings**. A packaged game does not read editor config files.

Saving from a Blueprint

Changes made on a settings page are saved automatically. When your own Blueprint changes a value, save it explicitly:

1. Call **Get Settings**.
2. Set the variable on the returned object.
3. Call **Save Settings** on that object.

The example below saves a grid size entered in an editor tool's window. The Level Design settings use personal storage for the current developer.



Check the result of Save Settings. If it returns false, read the log for the reason. See [Where the values are stored](#) for storage choices. For values changed by a packaged game, use **Storage = Saved**.

Call **Reload Settings** to discard unsaved changes. It restores the Blueprint's defaults and then applies stored values over them. A value that has never been saved returns to its default.

The section name

Each settings Blueprint stores its values under an ini section, such as `[BP_GameConfig]`.

By default, the plugin uses the Blueprint's name without the generated class suffix `_C`. Moving the asset to another folder keeps that section name. Renaming the Blueprint changes it.

For a name that stays the same after both moves and renames, set **Section Id** in Class Defaults. Choose a unique ID before first saving or releasing the settings, and keep it unchanged. A slash in the ID is stored as a dot.

If you change an ID after saving values, move the existing values to the new section in the ini file. Otherwise, the settings will read from the new section and will not find the old values.

Give each settings Blueprint its own section. Two Blueprints using the same section and config file can overwrite each other's values. This can happen with identical asset names or when a child Blueprint keeps its parent's Section Id. The plugin reports conflicting sections in the log.

The page

These options control the page:

- **Container** selects Project Settings or Editor Preferences.
- **Settings Category** selects the group, such as Game, Engine, Plugins or Editor. A new name creates a new group.
- **Settings Display Name** sets the title. If empty, the settings section name is used.
- **Settings Description** sets the text below the title. If empty, the Blueprint's description is used.

The page displays your variables. Edit its layout and storage options in **Class Defaults**.

Changes on the page are saved through the same process as **Save Settings**. The page's **Reset to Defaults** button performs the same reset as the node.

Set the original defaults in **Class Defaults** or in each variable's **Default Value** field. Saving, duplicating or packaging the Blueprint keeps those defaults separate from the current values on your machine.

Disable **Show a page in the settings panel** to hide the page while keeping the settings available to Blueprints.

Settings that build on other settings

To start from an existing settings Blueprint, right-click it in the Content Browser and choose **Create Child Blueprint Class**. The child has its parent's variables and defaults, and you can add variables of its own.

The child is a separate set of settings. It gets its own page, stores all of its values in its own section, including the ones it inherits, and Get Settings returns a different object for it.

The child also inherits the parent's page and section options. In its **Class Defaults**:

- Give it its own **Settings Display Name**, so the two pages do not share a title.

- If the parent sets a **Section Id**, give the child a different one. See [The section name](#).

Reset to Defaults

Reset Settings To Defaults restores the values set in the Blueprint's Class Defaults. This includes later edits to those defaults. It then saves the restored values to the ini file. The **Reset to Defaults** button on the settings page does the same thing.

Check the returned result:

- **True:** the defaults were restored and saved.
- **False:** the plugin had no recorded defaults, or saving the reset failed.

If saving fails, the values are reset only for the current session. Read the log for the reason.

Settings classes written in C++ do not have the separate record of Blueprint defaults used by this feature. Reset returns false for those classes.

Reacting to values

A settings Blueprint provides two events:

- **On Settings Loaded** runs after stored values have been read. Use it to apply settings to other systems, such as enabling a debug overlay. In a packaged game, values load before engine startup, but this event waits until the engine is ready.
- **On Settings Saved** runs after the values have been saved.

Other Blueprints can bind to **On Settings Changed** on the settings object. It notifies them after a load, save or reset, and after a change on the settings page. For example, a widget can update its display when the event fires instead of checking every frame.

Config variables on any object

Save Config Variables and **Reload Config Variables** also work on objects outside this plugin's settings classes. Use them for Blueprint variables marked **Config Variable**.

Packaging

The plugin includes the settings Blueprints it discovers when packaging the game. You do not need another asset to reference each settings Blueprint.

The plugin includes only settings Blueprints and the assets they use. A level that your game opens only by name must still be listed in the packaging settings. See [A level opened by name is missing from a build](#).

Packaging uses the defaults set in the Blueprint, not the current ini values on the build machine. This applies to builds made from the editor and from build scripts.

Open **Project Settings > Plugins > Blueprint Unchained — Project Settings** to control discovery. By default, the search covers all available content folders, including plugin content. You can limit it to selected paths or list classes in **Additional Settings Classes**. Listed classes are configured even if they are outside the search paths or automatic discovery is off.

Troubleshooting

The page is not in the settings panel

Compile the settings Blueprint and enable **Show a page in the settings panel**. Check **Container** to see whether the page belongs in Project Settings or Editor Preferences.

Get Settings has no variables to drag off

Select the specific settings Blueprint directly in **Settings Class**. For example, choose `BP_GameConfig`.

If the input wire only has the base Blueprint Project Settings class type, the result also has that base type. Select the specific class on the node, or cast the result to your settings class.

Save Settings returns false in the editor

Check that the variables have **Config Variable** enabled. At least one config variable is needed to save values. Read the log if saving still fails.

My settings work in the editor and are missing from a build

Check that the Blueprint is in a search folder or listed under **Additional Settings Classes**. If automatic discovery is disabled, only listed classes are configured.

A level opened by name is missing from a build

Unreal Engine packages the maps listed in **Project Settings > Packaging > List of maps to include in a packaged build**, the default maps from **Maps & Modes**, and everything those maps use. **Open Level (by Name)** names a level without referencing it, so that level is packaged only when it is listed.

With an empty list, the engine packages all project content instead, but only when nothing asks for specific content. In Unreal Engine 5.6 to 5.8, the **NNEDenoiser** plugin is enabled by default and asks for its own content, so most projects never reach that step. Blueprint Project Settings includes your settings Blueprints without such a request. Version 1.0.1 and earlier made one, which skipped that step in every project with a settings Blueprint.

To package the level, do one of the following:

- Add it to **List of maps to include in a packaged build**, in the advanced options of the **Packaging** section.
- Open it with **Open Level (by Object Reference)**. The engine follows that reference when packaging.

For other content that your game loads only by name or path, add its folder to **Additional Asset Directories to Cook**, or enable **Cook everything in the project content directory (ignore list of maps below)**.

Saving does nothing in a packaged game

Use **Storage = Saved** for values the game needs to write. Project Default cannot be written by a packaged game.

Also check **Config File**. Editor config files are available only in the editor. Read the log for the reason Save Settings returned false.

Values are back to the defaults after I moved the asset

Check whether you also renamed the Blueprint or changed **Section Id**. Moving an asset to another folder does not change the plugin's default section name.

Find the old section in the ini file and move its values to the section now used by the Blueprint. Use a fixed Section Id to keep the same section through future renames.

Reset to Defaults does nothing

Check the node's result and the log. Reset returns false if defaults were not recorded, as with a settings class written in C++, or if the restored values could not be saved.

Support

For help or bug reports, contact **tomasz.klin@gmail.com**.

Include your Unreal Engine version, plugin version, Storage option and Config File option. This manual covers plugin version 1.0.2.